

VISITOR MANAGEMENT SYSTEM USING SERVICENOW

Project Documentation

Objective

The objective of the Visitor Management System (VMS) project is to automate and streamline the process of managing visitor registrations within an organization. The application enables visitors to submit visit requests, allows administrators to approve or reject requests, records visitor check-in and check-out activities, sends automated email notifications, generates reports and dashboards, and provides a centralized platform for tracking visitor information. The system improves security, reduces manual paperwork, enhances visitor tracking, and ensures efficient management of visitor activities through an automated workflow built on the ServiceNow platform.

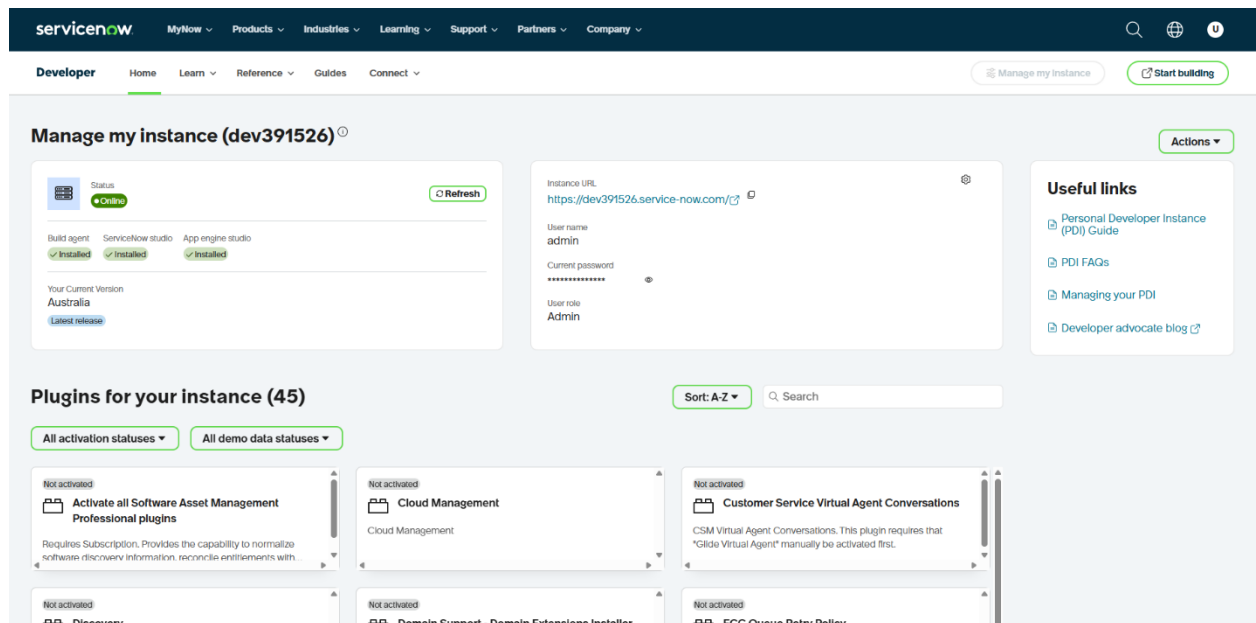
Milestone 1: Project Planning and Environment Setup

Objective

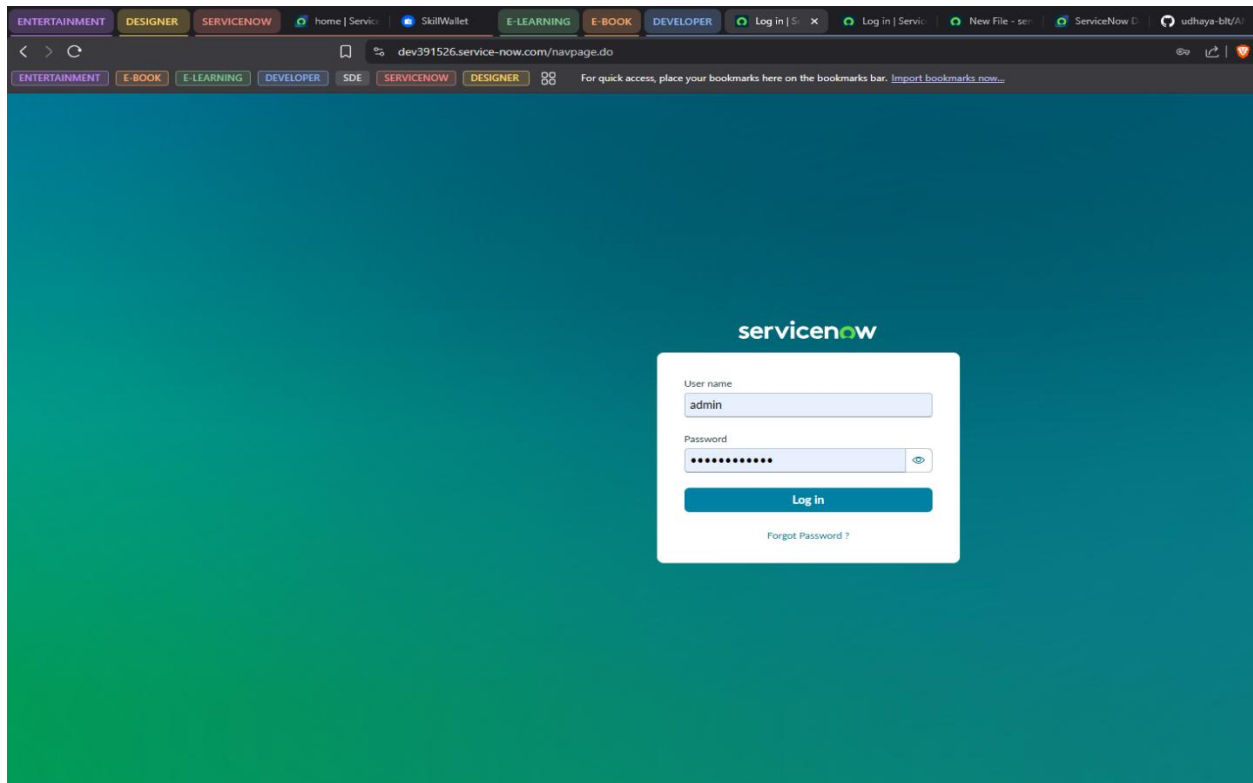
To prepare the ServiceNow development environment and establish the foundation for the Visitor Management System (VMS) application. This phase includes setting up the ServiceNow instance, creating the scoped application, and planning the database structure required for managing visitors and employee hosts.

Step 1: Create Personal Developer Instance (PDI)

1. Visit developer.servicenow.com.
2. Sign in or create a developer account.
3. Request a Personal Developer Instance.
4. Wait for instance activation.
5. Login using admin credentials.



The screenshot displays the ServiceNow Developer portal interface. At the top, the navigation bar includes links for MyNow, Products, Industries, Learning, Support, Partners, and Company. Below this, the 'Developer' section is active, showing links for Home, Learn, Reference, Guides, and Connect. The main content area is titled 'Manage my instance (dev391526)' and features a 'Status' section with a 'Refresh' button. The status section shows that the Build agent, ServiceNow studio, and App engine studio are all installed. Below this, the 'Your Current Version' is listed as 'Australia' with a 'Latest release' link. To the right, the 'Instance URL' is provided as <https://dev391526.servicenow.com/>, along with the 'User name' (admin), 'Current password' (masked), and 'User role' (Admin). A 'Useful links' section on the right provides links to the Personal Developer Instance (PDI) Guide, PDI FAQs, Managing your PDI, and the Developer advocate blog. The bottom section, 'Plugins for your instance (45)', shows a list of plugins with their activation status. The first three plugins are 'Activate all Software Asset Management Professional plugins', 'Cloud Management', and 'Customer Service Virtual Agent Conversations', all of which are currently 'Not activated'. The remaining three plugins are 'Discovery', 'Domain Support - Domain Extensions Installer', and 'ECC Queue Retry Policy', also marked as 'Not activated'.

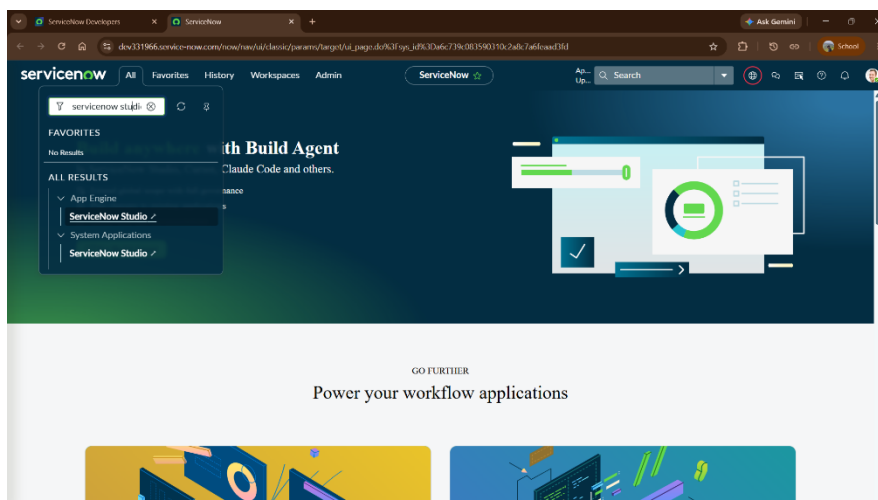


Step 2: Create Scoped Application

Navigation:

System Applications → Studio

1. Open Studio.
2. Select Create Application.
3. Enter:
 - Application Name: Visitor Management System
 - Scope: Auto-generated by ServiceNow
4. Click Create.



CREATE APP

Let's get started on your new app.

Add a name and description that define the purpose of your app.
You can also add a thumbnail image.

Name * ⓘ
Visitor Management System

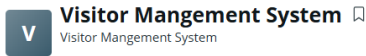
Description ⓘ
Describe this app

Scope * ⓘ
☒ Scoped
☐ Global

Browse or drag to upload

BMP, GIF, ICO, JPEG, JPG, PNG, SVG

Cancel Continue



Publish + Create ⋮

App type Scope Date created sys_id
Custom x_1877525_visito_0 2026-06-13 04:03:31 e4a1500c83210310c2a8c7a6feaad360

Learning Outcomes

- Understood ServiceNow architecture and application development lifecycle.
- Created and managed a scoped application.
- Configured the ServiceNow development environment.
- Learned ServiceNow Studio fundamentals.
- Understood table creation and relationship management.
- Gained experience in designing custom applications.

Milestone 2: Data Model Design

Objective

Design and implement the database structure for the Visitor Management System by creating custom tables, defining fields, and establishing relationships between employees and visitors.

Step 1: Create Employee Details Table

Navigation:

Studio → Create Application File → Table

Table:

Employee Details

Fields:

- Employee ID
- Employee Name
- Email
- Phone Number
- Department

- Designation
- Location
- Status

Purpose

The Employee table stores information about employees who can act as hosts for visitors.

ADD DATA

How do you want to create a table?

Tables contain the data that's available in an app. If you aren't sure where to start, we can help you create a new table from scratch. [Learn more about tables.](#)

CHOOSE HOW TO CREATE YOUR NEW TABLE STRUCTURE

Create new table

Create from an extensible table

Employee Details ▾

Data ▾

Forms ▾

Flows ▾

Policies and rules

Visitor Manage... ▾

↶

↷

Preview

⋮

Save

Table fields

+ Add new field

Filter fields

Filter

Column label *	Column name *	Type *	Reference	Max length	Default value	Display ↓
Employee Name	employee_name	Reference	User			☒
Updates	sys_mod_count	Integer				☐
Updated	sys_updated_on	Date/Time				☐
Class	sys_class_name	System Class Na...			javascript:current.g...	☐
Updated by	sys_updated_by	String		40		☐
Created	sys_created_on	Date/Time				☐
Employee ID	employee_id	String		40		☐
Phone Number	phone_number	Phone Number ...				☐
Location	location	String		40		☐
Designation	designation	String		40		☐

Add new field

Step 2: Create Visitor Details Table

Create another custom table.

Fields:

- Visitor Name
- Phone Number
- Email

- Company Name
- Visit Date (Date/Time)
- Employee Host
- Purpose of Visit
- Status
- Check In Time
- Check Out Time

Purpose

The Visitor table stores visitor details, visit schedules, approval status, and check-in/check-out information.

Column label *	Column name *	Type *	Reference	Max length	Default value	Display IF
Visitor Name	visitor_name	String		40		☒
Checked In Time	checked_in_time	Date/Time				☐
Updates	sys_mod_count	Integer				☐
Employee Host	employee_host	Reference	Employee Details			☐
Updated	sys_updated_on	Date/Time				☐
Checked Out Time	checked_out_time	Date/Time				☐
Updated by	sys_updated_by	String		40		☐
Created	sys_created_on	Date/Time				☐
Company Name	company_name	String		40		☐
Phone Number	phone_number	Phone Number ...				☐

Step 3: Create Reference Relationship

Reference Field

Employee Host

Reference Table

Employee

Relationship

One Employee → Many Visitors

Purpose

This relationship associates each visitor request with an employee host and enables administrators to track visitors assigned to a specific employee.

Step 4: Create Application Modules

Navigation

All → Application Menus & Modules

Modules Created

- Visitors
- Create Visitor Request
- Employees
- Host Visitor Requests

Purpose

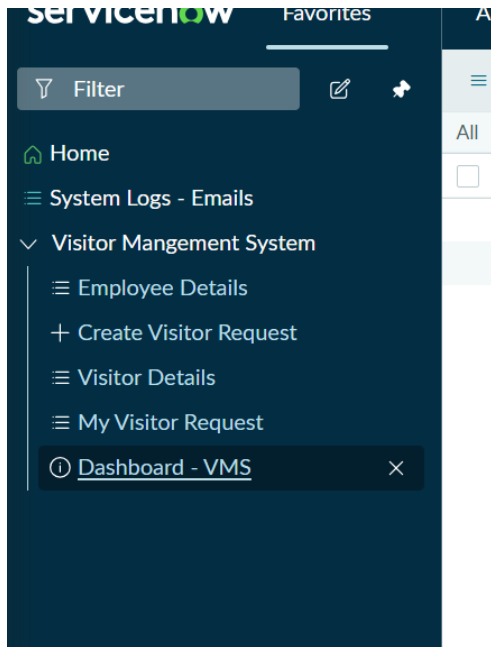
The application modules provide easy navigation to different functionalities of the Visitor Management System.

- **Visitors** – Displays all visitor records available in the system.
- **Create Visitor Request** – Allows users to submit a new visitor request.

- **Employees** – Displays employee records that act as visitor hosts.
- **Host Visitor Requests** – Allows users to view and track their submitted visitor requests.

Deliverable

Application modules were successfully created and configured to improve accessibility and navigation within the Visitor Management System.



Learning Outcomes

- Created custom tables.
 - Extended Task table.
 - Created relationships.
 - Built navigation modules.
-

Milestone 3: Business Logic and Security

Objective

Implement business logic, form customization, validation rules, and user interface configurations to automate visitor management processes and improve user experience.

Step 1: Configure Form Layout

Navigation:

Open Visitor Form

Right Click

Configure

Form Layout

Arrange fields in a logical order to improve usability and data entry efficiency.

Step 2: Create UI Policies

Purpose

Dynamically control field visibility and improve the user experience by displaying relevant fields based on the visitor's status.

Navigation

System UI → UI Policies

Create New UI Policy

UI Policy 1: Display Check In Time

Table

Visitor

Condition

Status = Checked In

OR

Status = Checked Out

UI Policy Action

- Check In Time → Visible = True
- Check In Time → Read Only = True

Purpose

Displays the Check In Time field after the visitor has been checked in and prevents users from modifying the recorded value.

UI Policy

Show Check in time

Update

Delete

Table

Visitor_details [x_1877525_visit...

Application

Visitor Mangement System

Active

☒

* Short description

Show Check in time

When to Apply

Conditions

Add Filter Condition

Add OR Clause

Status

is

Checked In

AND

OR

×

or

Status

is

Checked Out

×

Update

Delete

Related Links

[Advanced view](#)
[Run Point Scan](#)
[\[SN Utils\] Versions \(1\)](#)
[\[SN Utils\] Versions \(1\)](#)

UI Policy Actions (1)

UI Policy Related List Actions

for text

Search

Actions on selected rows...

New

UI policy = Show Check in time

Field name	Mandatory	Visible	Read only
checked_in_time	Leave alone	True	True

1 to 1 of 1

UI Policy Action

checked_in_time

Update

Delete

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy

Show Check in time

Application

Visitor Mangement System

Table

Visitor_details [x_1877525_visito_0_visitor_details]

* Field name

Checked In Time

Mandatory

Leave alone

Visible

True

Read only

True

Clear the field value

☐

Update

Delete

[SN Utils] Versions (1)

UI Policy 2: Display Check Out Time

Table

Visitor

Condition

Status = Checked Out

UI Policy Action

- Check Out Time → Visible = True
- Check Out Time → Read Only = True

Purpose

Displays the Check Out Time field only after the visitor has been checked out and prevents modification of the recorded timestamp.

UI Policy

Show Check Out Time

UpdateDelete

Table

Visitor_details [x_1877525_visit...

Application

Visitor Mangement System

Active

* Short description

Show Check Out Time

When to Apply

Conditions

Add Filter Condition

Add OR Clause

Status

is

Checked Out

ANDOR

Update

Delete

Related Links

[Advanced view](#)
[Run Point Scan](#)
[\[SN Utils\] Versions \(1\)](#)

UI Policy Actions (1)

UI Policy Related List Actions

for text

Search

Actions on selected rows...

New

UI policy = Show Check Out Time

	Field name	Mandatory	Visible	Read only
☐	checked_out_time	Leave alone	True	True

1 to 1 of 1

UI Policy Action

checked_out_time

UpdateDelete

UI policy-actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy

Show Check Out Time

Application

Visitor Mangement System

Table

Visitor_details [x_1877525_visit...

* Field name

Checked Out Time

Mandatory

Leave alone

Visible

True

Read only

True

Clear the field value

Update

Delete

[SN Utils] Versions (1)

[SN Utils] Versions (1)

Step 3: Create Custom Role

Navigation:

sys_user_role.list

Role 1:

x_1877525_visit_0.admin

Assign role to Security.

Role 2:

x_1877525_visit_0.user.

Assign role To Employees.

Role

x_1877525_visito_0.admin

Update

Delete

* Suffix

admin

Application

Visitor Mangement System

Elevated privilege

☐

Name

x_1877525_visito_0.admin

Assignable by

Description

Default admin role

Update

Delete

[SN Utils] Versions (1)

[SN Utils] Versions (1)

Contains Roles

Applications with Role (1)

Modules with Role (5)

Custom Tables

for text

Search

New

Edit...

Role = x_1877525_visito_0.admin

Contains

Role

x_1877525_visito_0.user

Update

Delete

* Suffix

user

Application

Visitor Mangement System

Elevated privilege

☐

Name

x_1877525_visito_0.user

Assignable by

Description

Default user role

Update

Delete

[SN Utils] Versions (1)

Contains Roles

Applications with Role (1)

Modules with Role (5)

Custom Tables

for text

Search

New

Edit...

Role = x_1877525_visito_0.user

Contains

Step 4: Create UI Actions

Purpose

Provide administrators with quick actions to manage visitor requests directly from the Visitor form.

UI Actions Created

- Approve
- Reject
- Check In
- Check Out

Configuration

Each UI Action updates the visitor status and performs the required business logic.

Purpose

These actions automate the visitor approval and visit tracking process, reducing manual effort and improving operational efficiency.

Client Script

Vaildate Visit date

Update

Delete

Name

Vaildate Visit date

Table

Visitor_details [x_1877525_visit...

UI Type

All

Type

onSubmit

Application

Visitor Mangement System

Active

☒

Inherited

☐

Global

☒

Description

Messages

Script

1

function onSubmit() {

2

3

var visitDate = g_form.getValue('visit_date');

4

5

if (!visitDate)

6

return true;

7

8

var selectedDate = new Date(visitDate);

9

var now = new Date();

10

11

// Past date/time validation

12

if (selectedDate < now) {

13

g_form.addErrorMessage("Visit date and time cannot be in the past.");

14

return false;

15

}

16

17

// Working hours validation (9 AM - 6 PM)

18

var hour = selectedDate.getHours();

19

Learning Outcomes

- Learned to create and configure UI Policies.
- Implemented dynamic field visibility and read-only controls.
- Created Client Scripts for date and time validation.
- Applied business logic to enforce working-hour restrictions.
- Developed UI Actions for visitor approval, rejection, check-in, and check-out processes.
- Improved data quality through client-side validation.
- Enhanced user experience by automating visitor management workflows.

Milestone 4: Notifications and Dashboard Monitoring

Objective

Improve communication and visitor tracking by sending automated email notifications and providing real-time dashboard insights.

Step 1: Create Email Notifications Using Flow Designer

Purpose

Automatically notify stakeholders whenever a visitor request is submitted or its status changes.

Navigation

Flow Designer → Create Flow

Flow 1: Visitor Request Submitted

Trigger

Visitor record created

Recipient

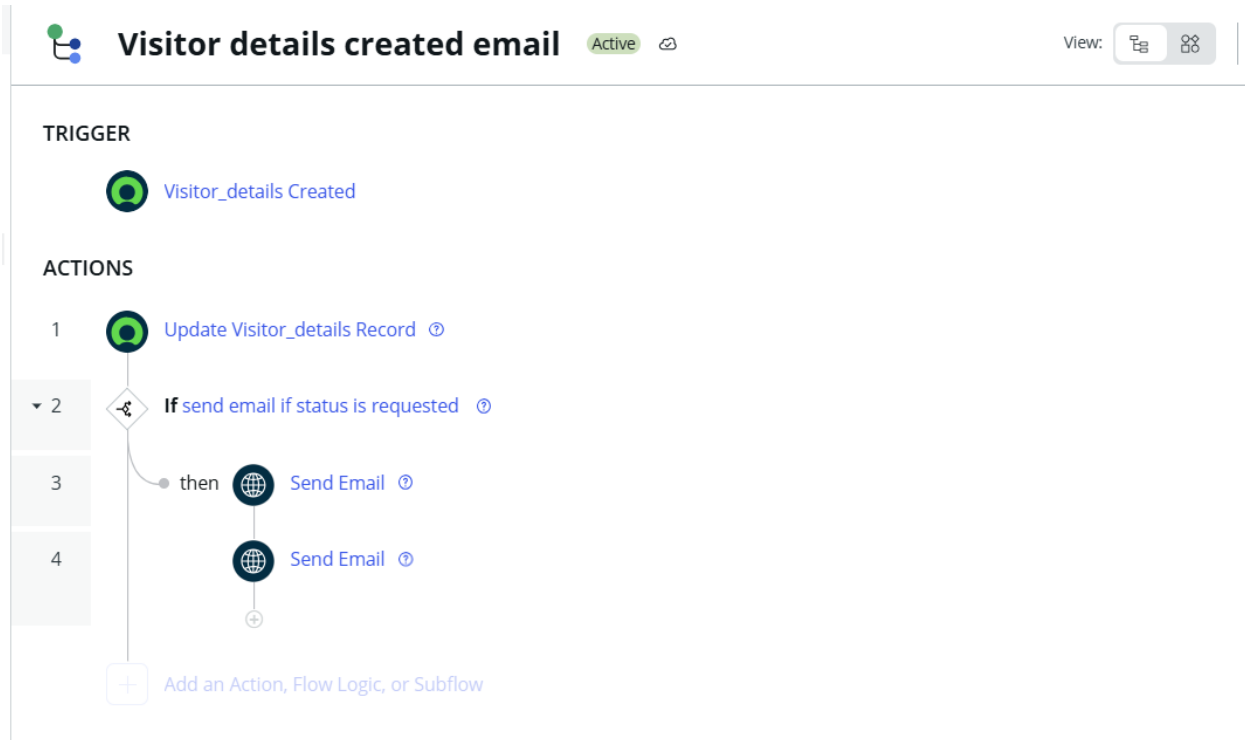
Employee Host, Visitor

Email Contents

- Visitor Name
- Company Name
- Visit Date
- Purpose of Visit
- Request Status

Result

The employee host receives an email notification whenever a new visitor request is submitted.



Step 2: Create Status Update Notifications

Trigger

Visitor Status Updated

Recipients

- Employee Host
- Visitor

Email Contents

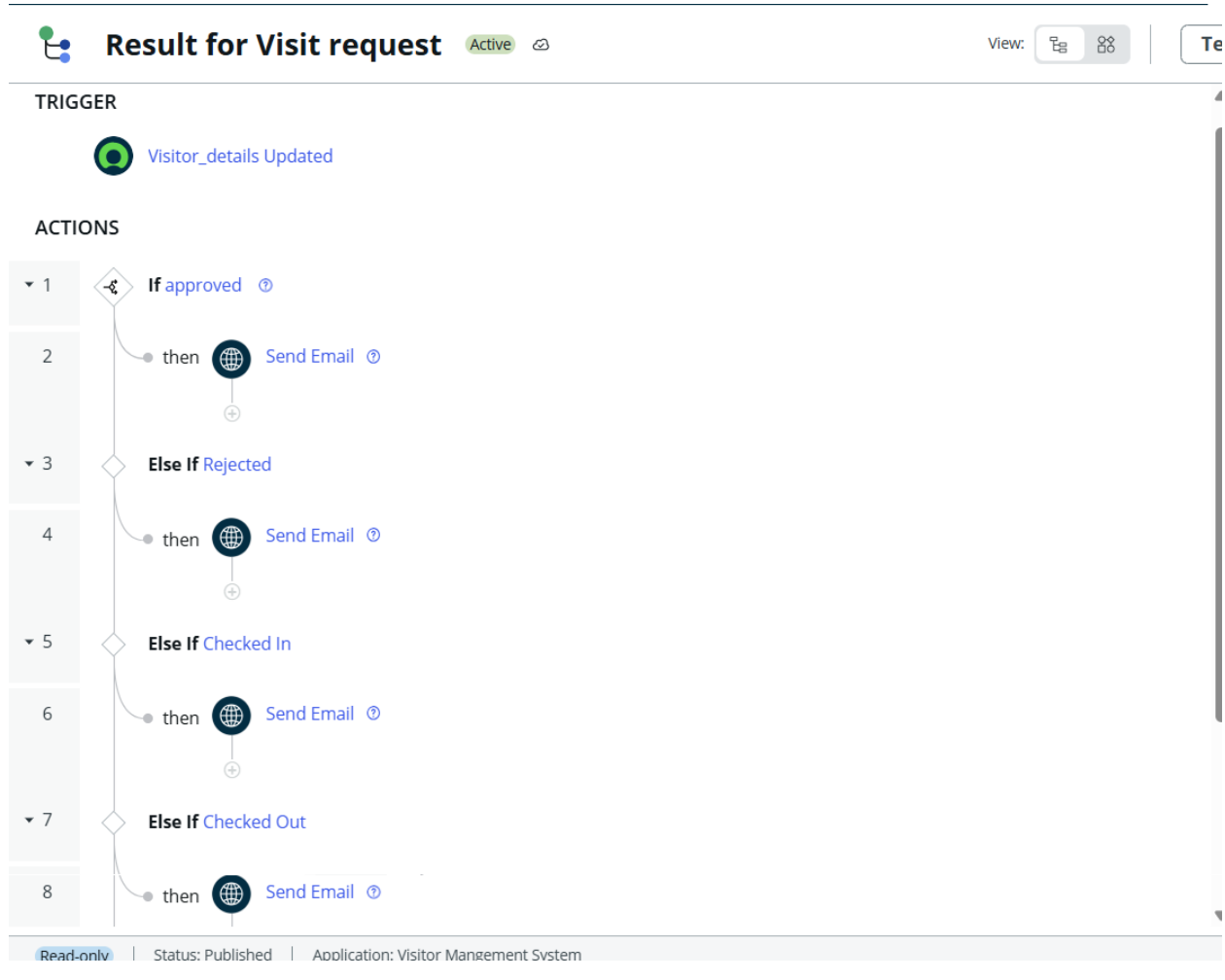
- Visitor Name
- Visit Date
- Current Status
- Employee Host

Status Notifications

- Approved
- Rejected
- Checked In
- Checked Out

Result

Relevant users receive automated email updates whenever the visitor request status changes.



Step 3: Create Visitor Management Dashboard

Purpose

Provide administrators with a centralized view of visitor activities and request statuses.

Navigation

Platform Analytics → Dashboards

Create New Dashboard

Dashboard Name

Visitor Management System Dashboard

Widgets Added

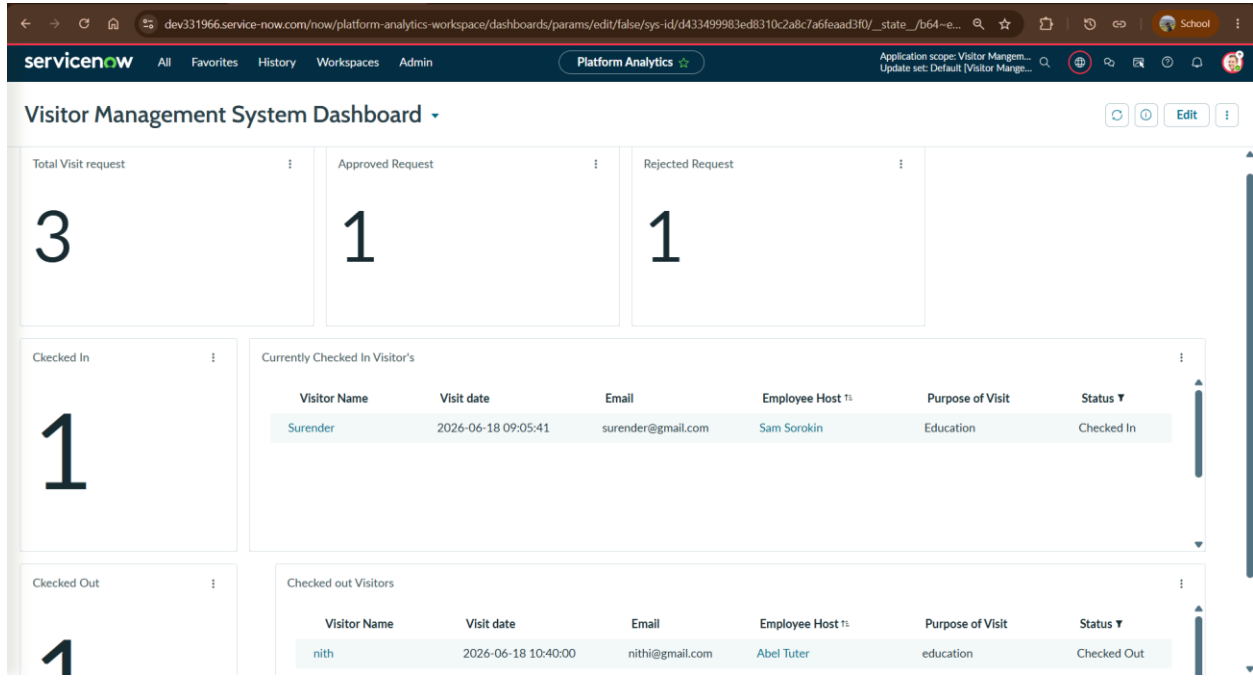
- Total Visit Requests
- Approved Requests
- Rejected Requests
- Checked In Visitors Count
- Currently Checked In Visitors List
- Checked Out Visitors Count
- Checked Out Visitors List

Purpose

The dashboard provides real-time visibility into visitor requests, approvals, rejections, check-ins, and check-outs, enabling efficient monitoring and management of visitor activities.

Result

Administrators can quickly track visitor statistics and view active and completed visits from a single dashboard.



Step 4: Verify Email Notifications

Purpose

Test and verify that email notifications are sent successfully when visitor-related events occur.

Testing Performed

- Submitted a new visitor request.
- Updated visitor status to Approved.
- Updated visitor status to Rejected.
- Performed Check In operation.
- Performed Check Out operation.

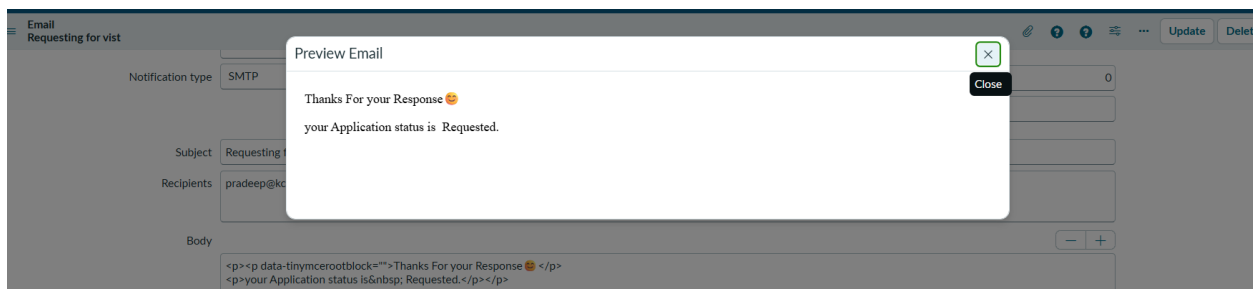
Verification

Confirmed that email notifications were delivered to the visitor's registered email address with the correct visitor details and status information.

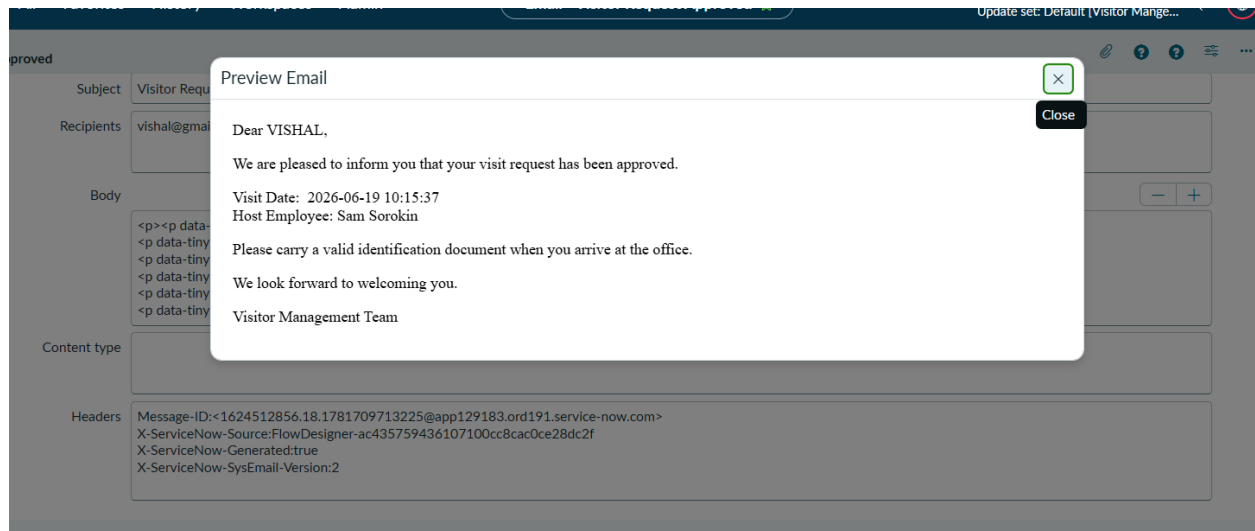
Result

Email notifications were successfully triggered through Flow Designer and delivered to visitors at each stage of the visitor management process.

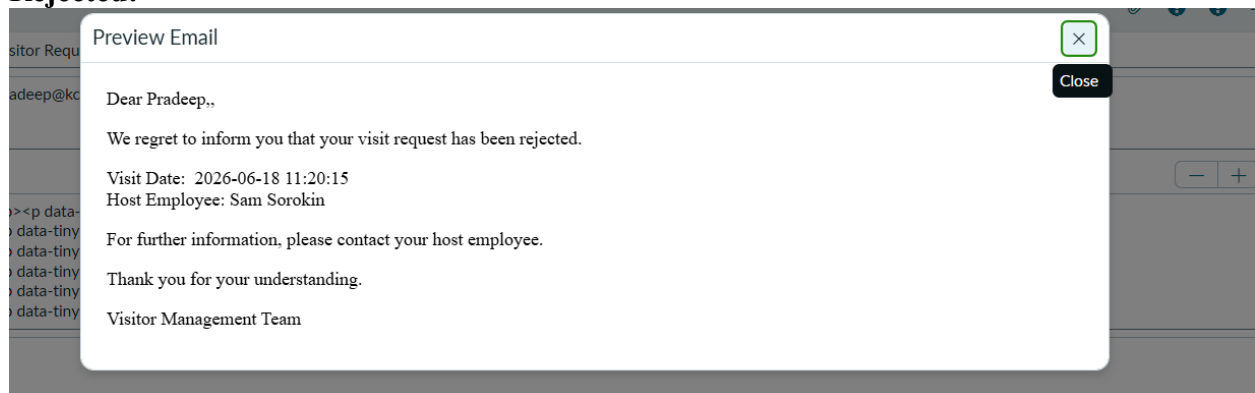
Requested:



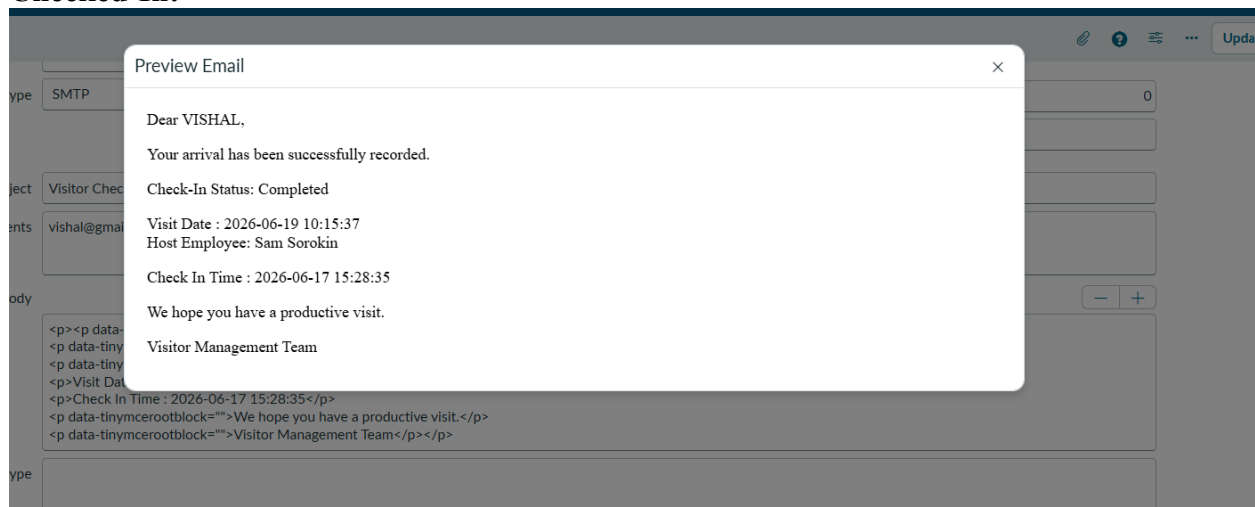
Approved:



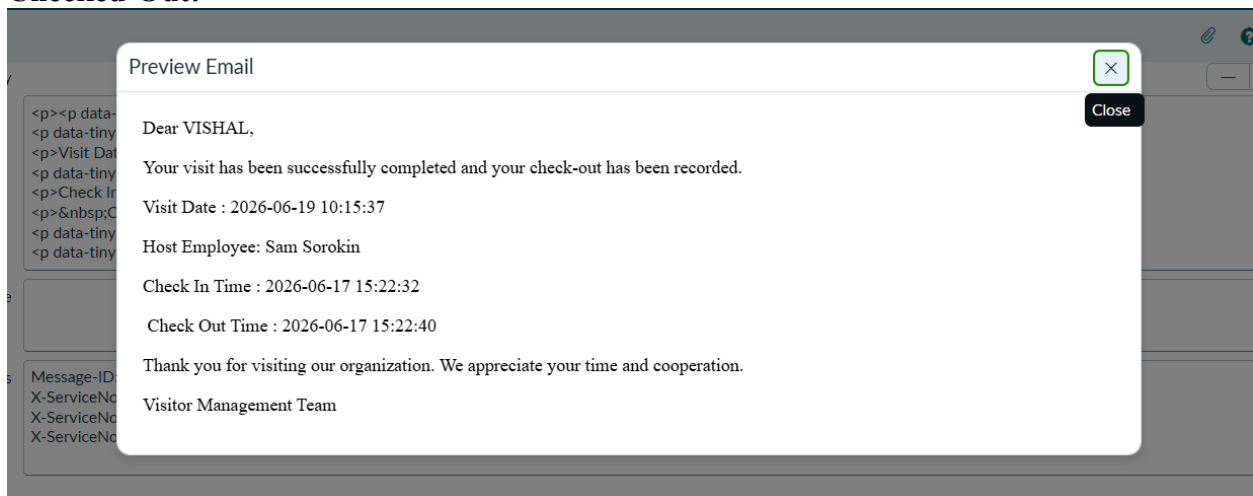
Rejected:



Checked-In:



Checked-Out:



Learning Outcomes

- Created and configured Flow Designer email notifications.
- Verified email delivery and notification processing.
- Designed and developed a Visitor Management Dashboard.
- Configured dashboard widgets for visitor monitoring.
- Learned real-time tracking and visualization of visitor activities.
- Improved reporting and analytics through dashboard insights.

Final Testing Checklist

- ✓ Scoped Application
- ✓ Employee Table
- ✓ Visitor Table
- ✓ Reference Relationship
- ✓ Application Modules
- ✓ Form Layout Configuration
- ✓ UI Policies
- ✓ Client Scripts
- ✓ Working Hours Validation
- ✓ Approve UI Action
- ✓ Reject UI Action
- ✓ Check In UI Action
- ✓ Check Out UI Action
- ✓ Flow Designer Notifications
- ✓ Visitor Management Dashboard
- ✓ End-to-End Testing
- ✓ Application Export

Conclusion

The Visitor Management System (VMS) successfully automates the process of managing visitor requests using the ServiceNow platform. The application enables visitor registration, approval and rejection of requests, visitor check-in and check-out tracking, automated email notifications, and dashboard-based monitoring. The project demonstrates key ServiceNow concepts including scoped application development, custom tables, reference relationships, client scripts, UI policies, UI actions, Flow Designer automation, and dashboard creation. The solution improves visitor tracking, reduces manual effort, enhances communication, and provides real-time visibility into visitor activities.

Future Enhancements

- Multi-level approval workflow.
- Visitor badge generation and printing.
- QR Code based visitor check-in.
- Visitor self-service portal.
- Mobile-friendly interface.
- Visitor history and audit tracking.
- Advanced dashboard analytics and trends.
- Integration with attendance or access control systems.
- Automated visitor pass expiration notifications.
- AI-assisted visitor insights and analytics.